Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with 'A' Grade (CGPA : 3.18)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)

EXPERIMENT NO. 4

Name : Devyani Kamat SAP ID : 60017240102

AIM:

To create standard API Route Handlers (route.ts) alongside native React Server Actions to manage full CRUD data mutations (GET, POST, PUT, DELETE) without traditional API fetch boilerplates.

SOFTWARE REQUIRED:

- **Operating System:** Linux / macOS / Windows 10/11
- **Node.js runtime environment:** (v18.17.0 or higher)
- **Package Manager:** npm or pnpm
- **Code Editor:** Visual Studio Code

THEORY:

- **Next.js App Router Route Handlers** (route.ts)**:** Route Handlers allow developers to create custom request handlers for a given route using the Web Request and Response APIs. They replace the legacy pages/api endpoints and support standard HTTP verbs (GET, POST, PUT, DELETE, PATCH). Route Handlers execute exclusively on the server, ensuring secure database access and external API secret management.
- **React Server Actions ('use server'):** Server Actions are asynchronous JavaScript functions that execute on the server. They can be called directly from Client or Server Components, or invoked inside HTML <form> action attributes. By leveraging Server Actions, developers can perform mutations directly from the UI layer without manually writing client-side fetch() requests, useEffect hooks, or managing manual loading state boilerplates.
- **Route Cache Invalidation** (revalidatePath)**:** When mutations occur on the server (via Server Actions or Route Handlers), Next.js's aggressive client-side Data Cache and Router Cache may serve stale data. The revalidatePath('/route') function purges the cached data for a specific route path, forcing a silent server re-render of the component tree and serving fresh data seamlessly.

STEPS / ALGORITHM:

Step 1: Create an In-Memory Data Store (app/lib/db.ts)

Establish a simple shared state array to simulate a backend database for CRUD operations.

```
export interface BlogItem {
  id: string;
  title: string;
  content: string;
}

// In-memory data store for demonstration
```

Shri Vile Parle Kelavani Mandal's
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING
(Autonomous College Affiliated to the University of Mumbai)
NAAC Accredited with "A" Grade (CGPA : 3.18)

Department of Artificial Intelligence and Machine Learning
B.Tech. Sem: V Subject: Full Stack Development Using NextJs (DJS23AMD302L)





Observation : API route handlers alongside native React server actions managed the full CRUD data mutations.

conclusion : API route handlers allow developers to create custom request handlers for a given route using Web Request & Response APIs.



CONCLUSION:

Standard API Route Handlers (route.ts) and native React Server Actions were successfully constructed. The experiment verified full HTTP CRUD operations, eliminated traditional client-side API fetch boilerplates via Server Actions, and demonstrated automatic cache revalidation using revalidatePath.

POST LAB QUESTION:

Q: What are the security and performance benefits of using Server Actions over client-side fetch() calls to Route Handlers?

A:

1. **Reduced Bundle Size & Zero Boilerplate:** Server Actions eliminate the need to ship client-side fetching libraries, manual useState loading/error boilerplate code, and useEffect hooks to the browser, significantly reducing the JavaScript bundle size.

2. **Security & Server-Only Execution:** Server Actions execute exclusively on the server. Database credentials, secret keys, and complex validation logic are never exposed to the client bundle or network payload.

3. **Progressive Enhancement:** Forms backed by Server Actions can function even if JavaScript is slow to load or disabled on the client browser, whereas standard fetch() callbacks depend entirely on client-side JavaScript execution.